School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	B00161706
Student Name:	Alexander Chiper

Module Information

Module Title:	Web Framework Development
Module Code:	COMP H3037

Assessment Information

Assessment Title:	Project

Declaration

☑ I confirm that I have read and understood the University's Academic Integrity Policy.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/

☑ I have completed the TU Dublin AI Driving License theory test.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/lta/lta-resources/artificial-intelligence-driving-license/

☑ I declare that this assessment submission is my own work.

☑ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student Signature: Alexander Chiper

Date:4/18/2026

Web Framework Development Project:

[Agriculture – Recruiting Candidate-Skills]

AgriHiredCulture

Submitted by: Alexander Chiper, B00161706

Table of Contents . . .

1. Business Understanding

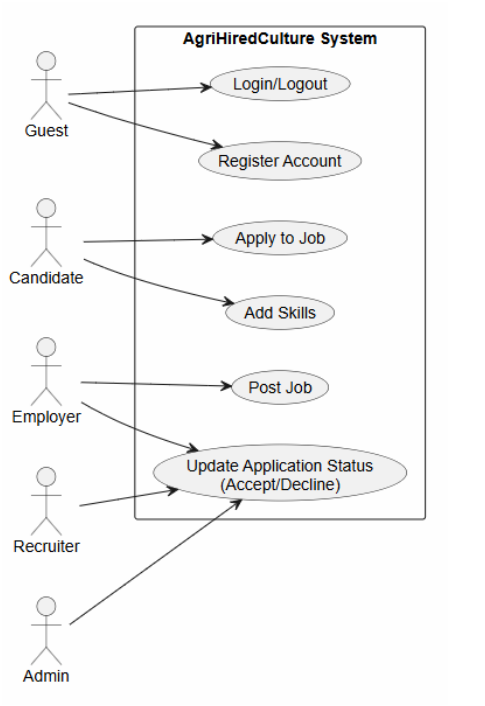
AgriHiredCulture is a web recruitment application made for the agricultural sector. The application connects employers and businesses to employees that have relevant agricultural skills

Business Requirements

The application must have:

- Provide a system for agricultural job recruitment
- Allow candidates to create profiles and manage their skills
- Employers can manage job postings
- Allow employers to search for employees based on their skills
- Support job application functionality
- Be role supported (Employee/Candidate, Employer, Recruiter and Admin)
- Have secure login and Registration
- Store and manage data efficiently using a relational database
- Have a user-friendly interface

2. Use Cases



Login/Logout

Guest initiates it. Allows new users to get access to their account in the system. This is the entry point for Candidate, Employer, etc.

Register Account

Registration is performed by the Guest it creates an account; you can register as a candidate, recruiter or Employer the boss, you have access to the sections provided.

Add Skills

Add skills is performed by the candidate, candidates can choose which skills they have form list of skills to strengthen their profile, this information helps them land jobs

Post Job

Posting Jobs is carried out by the Employer. Employers create job listings the requirements needed for the job

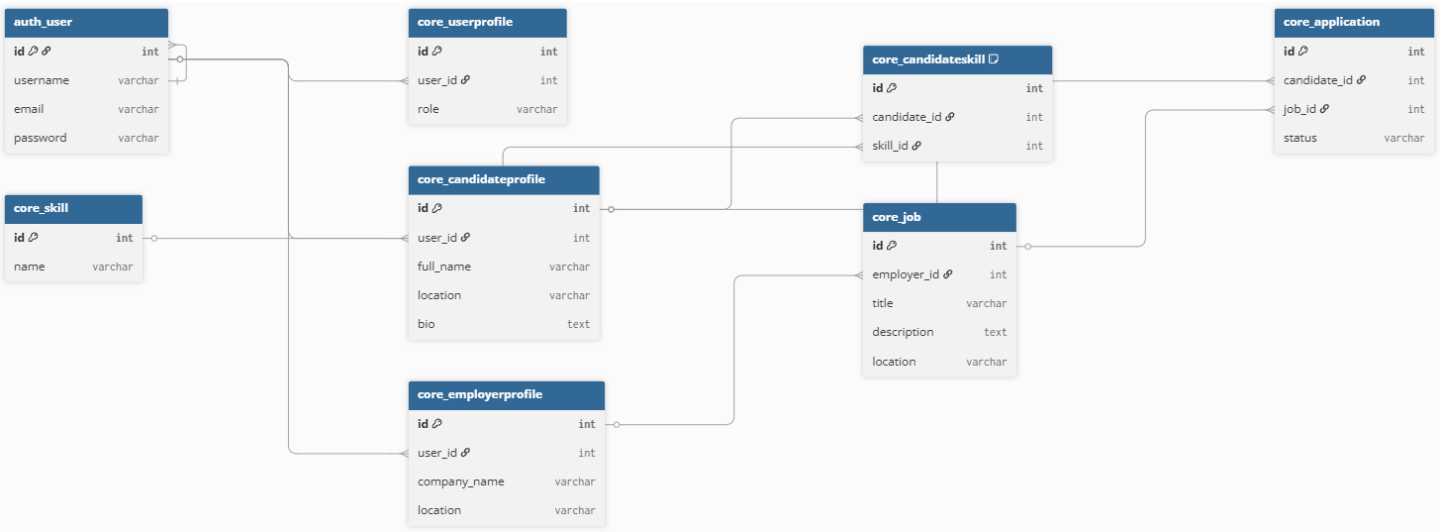
Apply to Job

Applying for jobs is done by the Candidate. Candidates submit applications to jobs posted by employers.

Update Application Status

Shared by Employer, Recruiter, and Admin. Allows these users to accept applications and decline applications.

3. Database Design



4. Django Framework Implementation

M – Model

The main entities and relationships for the app is defined by the model layer, which is implemented in core/models.py, UserProfile adds a role to a Django user. The link between CandidateProfile and User is one-to-one. The link between EmployerProfile and User is one-to-one. Unique skill names are stored in the skill. CandidateSkill prevents duplicate skills for each applicant by linking CandidateProfile and Skill with unique together. The job is part of an employer profile. The application associates a CandidateProfile with a job that has the status pending to be accepted or declined.

This structure is consistent and with the database design in which profile information is divided into profile tables, user identification is in Django User, and transactional entities use foreign keys to refer to those profiles.

V – View

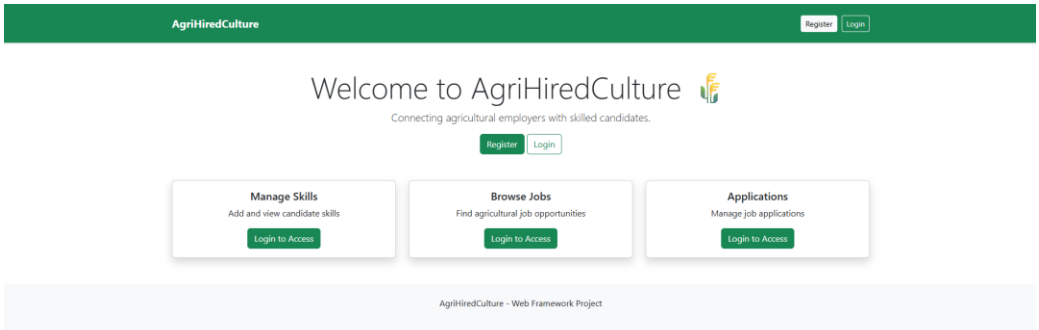
It's a role-based access control and function is also role-based views and are used in core/views.py to construct the view layer, Django's built-in login view is in core/urls.py, along with the registration and logout views. Features for candidates include add_candidate_skill, skill list, and add application. Recruiter/admin overview, a list of applications with more visibility. Role checks and @login_required are used to impose access control.

T – Template

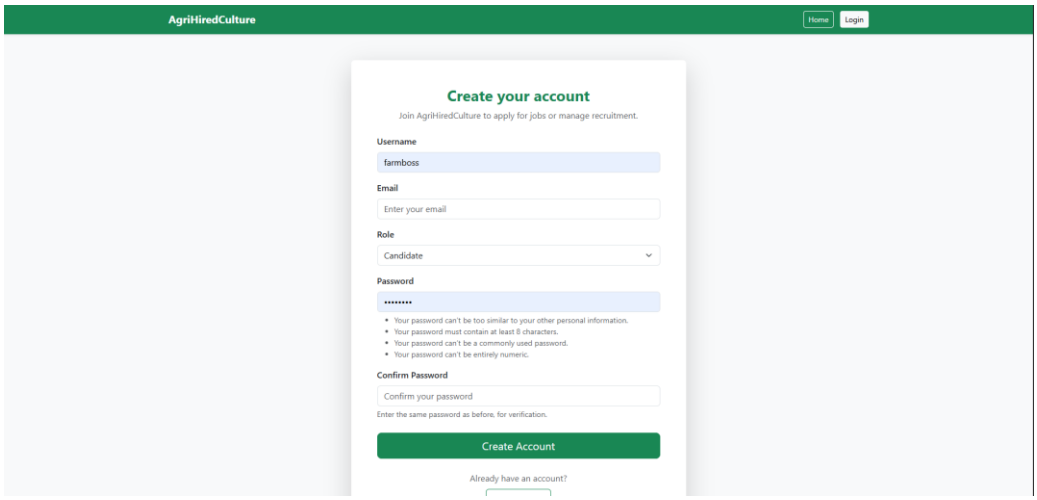
This is a role specific user interface, and it is rendered with templates located in core/templates/core/. home.html, login.html and register.html, add_candidate_skill.html and skill_list.html, add_job.html and job_list.html, add_application.html and application_list.html.

The template section shows dynamic records and role-based actions using Django template and context data from views.

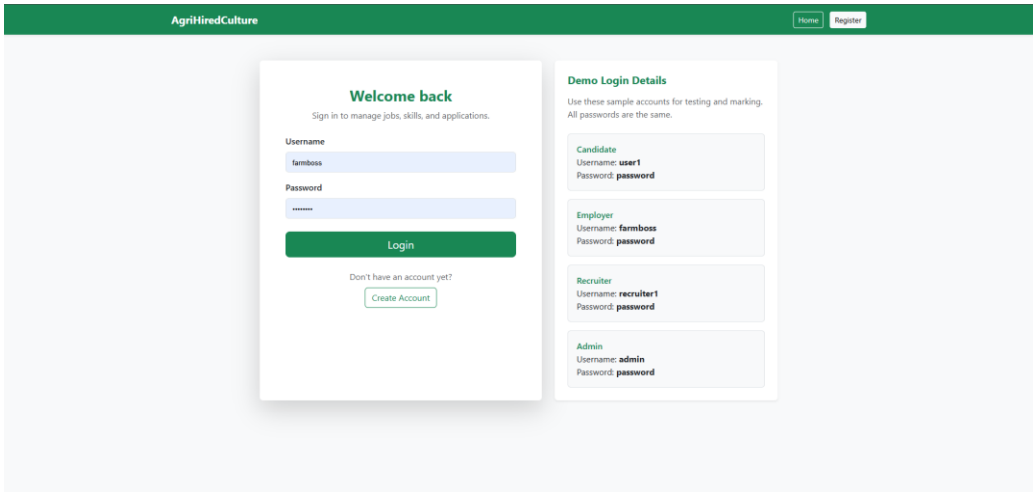
Home Page:



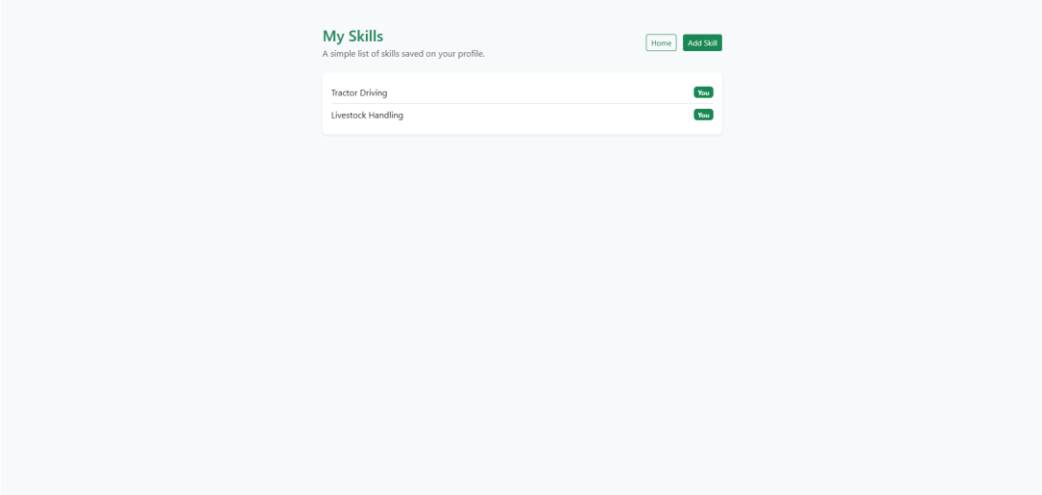
Register Page:



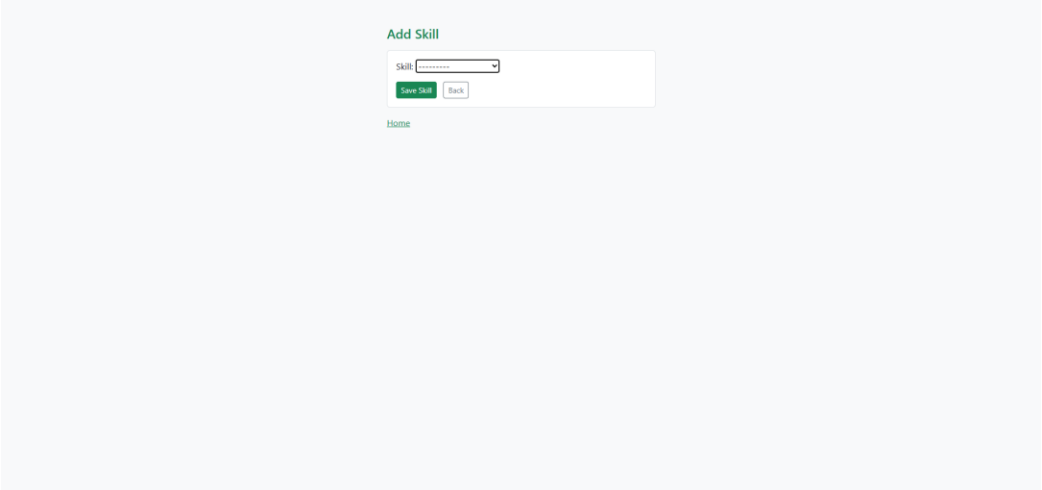
Login Page:



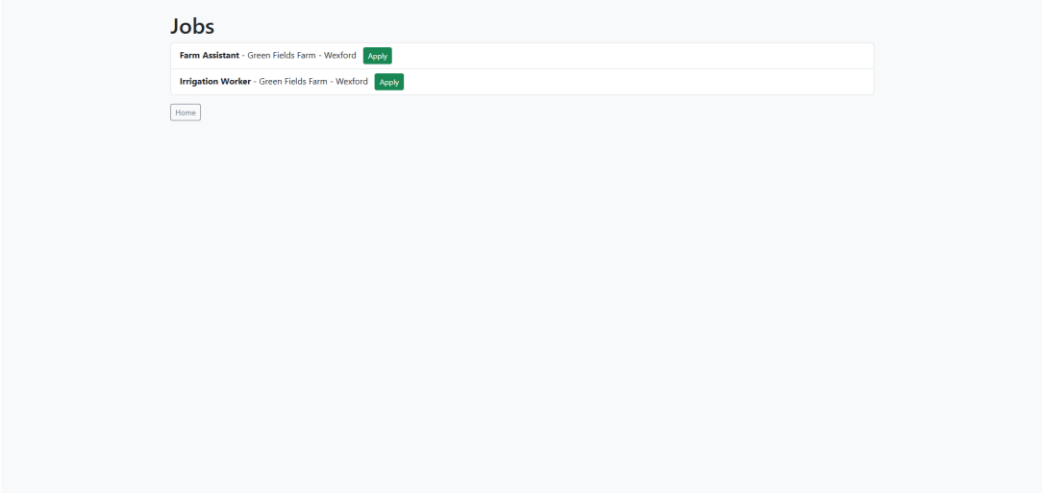
Candidate And Skill Page:



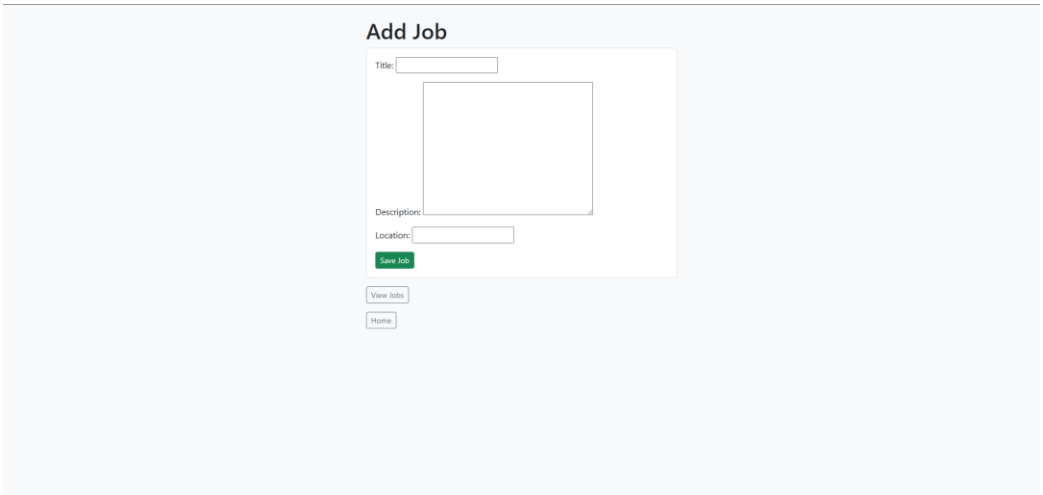
Candidate Add Skill Page:



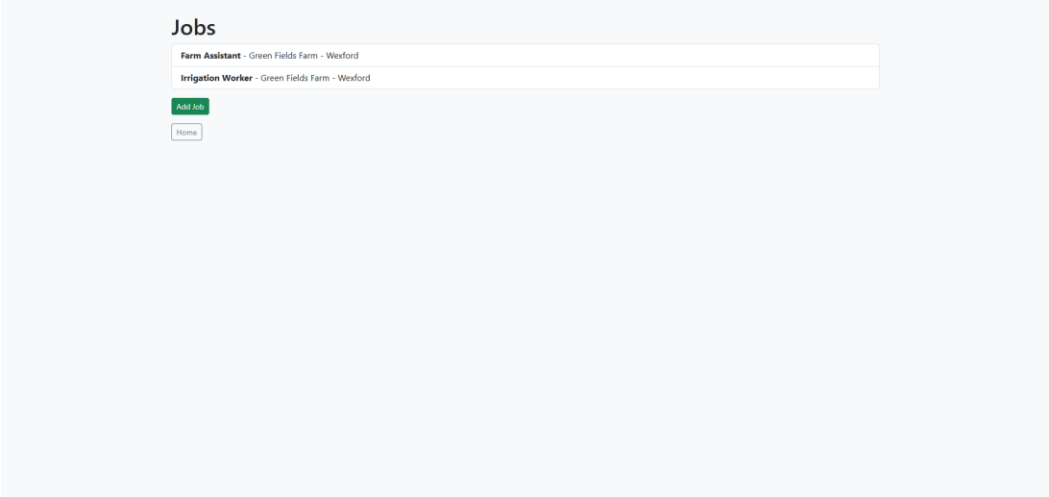
Candidate Apply Page:



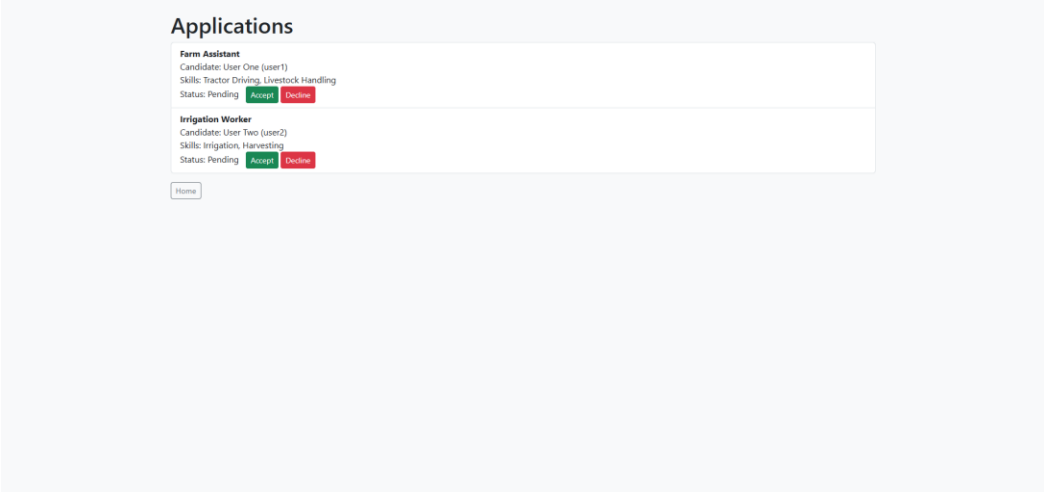
Employer Add Job Page:



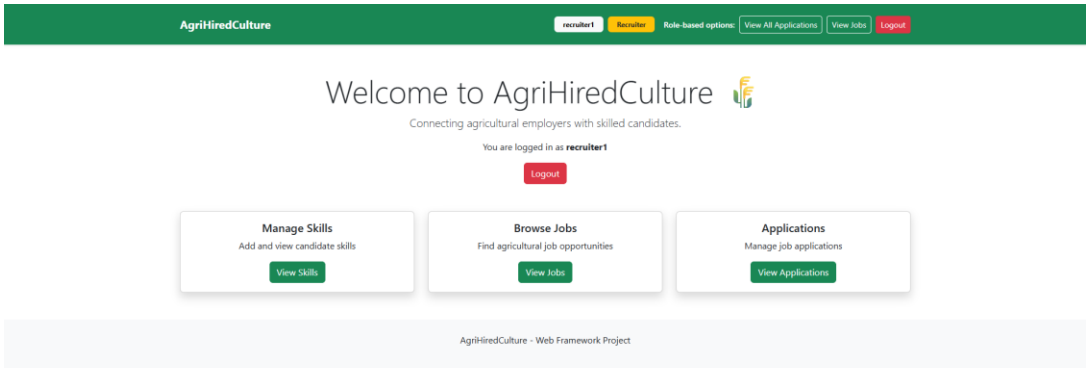
Job List Page:



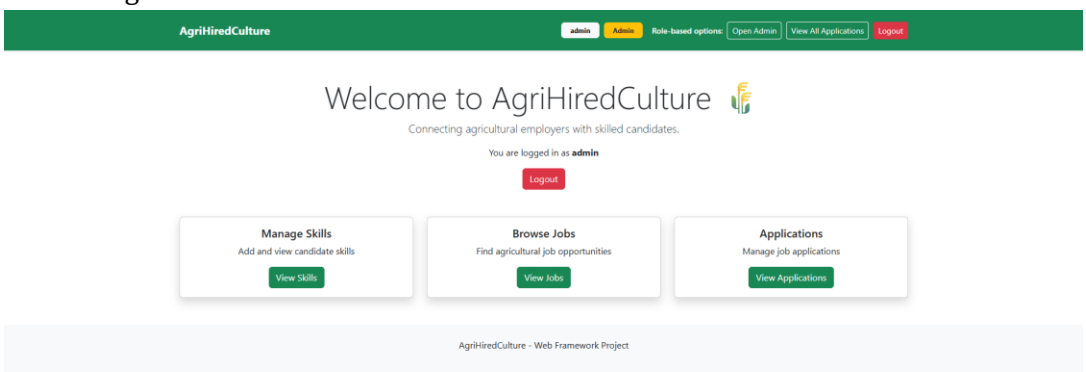
Employer application List Page:



Recruiter Page:



Admin Page:



5. Project Testing

Testing is implemented in core/tests.py:

Unit tests

Focus on data behaviour:

- ModelTests such as account creation etc
- Validates object creation

Integration tests

Focus on interactions across the application:

- IntegrationTests verifies end-to-end model interactions
- ViewAndRegistrationTests validates page rendering and registration

Use case tests:

- candidate adds skill and applies to job

- employer posts job and sees only relevant applications
- recruiter sees all applications
- authorization rules
- duplicate prevention and status update workflow

```
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\COLLEGE\Year 3 Sem 2\Web Frameworks\AgriHiredCulture>python manage.py test
Found 30 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
......Database seeded successfully.
.C:\Users\Alex\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\django\core\handlers\base.py:62: UserWarning: N
o directory at: C:\COLLEGE\Year 3 Sem 2\Web Frameworks\AgriHiredCulture\staticfiles\
  mw_instance = middleware(adapted_handler)
.........................
----------------------------------------------------------------------
Ran 30 tests in 35.366s

OK
Destroying test database for alias 'default'...

C:\COLLEGE\Year 3 Sem 2\Web Frameworks\AgriHiredCulture>
```

- All automated tests passed successfully

6. Media Sources

https://agrihiredculture.onrender.com/

https://github.com/Alexchiper1/AgriHiredCulture

By submitting your project for assessment, you agree to the following:
"The material contained in this assignment is my own original work, except where work is clearly identified and duly acknowledged. No aspect of this assignment has been previously submitted for assessment in any other unit or course."